

test cases		1	2	3	4	5	6	7
percentage of bridges - <0.1% - 1 0.1-5% - 2 5%-50% - 3 >50% - 4								
number of components - 1 - 1 component TLE - a lot of components		2.00	2.00	1.00	2.00	2.00	1.00	
type		TLE	TLE	1.00	TLE	TLE	1.00	
average degree TLE - probably high 3- (special case) there is no shortest path from any vertex to any other that is >= 3		1		2	1		2	
		5.00	8.00	TLE	5.00	8.00	TLE	

test cases		7	8	9	10	11	12	13	14	15	16	17
percentage of bridges - <0.1% - 1 0.1-5% - 2 5%-50% - 3 >50% - 4												
number of components - 1 - 1 component TLE - a lot of components		1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
type		1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
average degree TLE - probably high 3- (special case) there is no shortest path from any vertex to any other that is >= 3		5.00	5.00	5.00	6.00	6.00	6.00	6.00	6.00	5.00	5.00	

test cases	17	18	19	20	21	22
percentage of bridges - <0.1% - 1 0.1-5% - 2 5%-50% - 3 >50% - 4						
number of components - 1 - 1 component TLE - a lot of components	1.00	1.00	1.00	1.00	1.00	1.00
type						
average degree	1.00	1.00	1.00	1.00	1.00	1.00
TLE - probably high 3- (special case) there is no shortest path from any vertex to any other that is ≥ 3						
	2.00	2.00	2.00	2.00	2.00	2.00

test cases	22	23	24	25	26	27	28	29	30	31	32
percentage of bridges - <0.1% - 1 0.1-5% - 2 5%-50% - 3 >50% - 4											
number of components - 1 - 1 component TLE - a lot of components	4.00	4.00	4.00	4.00	4.00	4.00	4.00	4.00	4.00	4.00	4.00
type											
average degree	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
TLE - probably high 3- (special case) there is no shortest path from any vertex to any other that is ≥ 3											
	2.00	2.00	2.00	2.00	2.00	2.00	2.00	2.00	2.00	2.00	2.00

test cases	32	33	34	35	36	37	38	39	40
percentage of bridges - <0.1% - 1 0.1-5% - 2 5%-50% - 3 >50% - 4									
number of components - 1 - 1 component TLE - a lot of components	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
type									
average degree	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
TLE - probably high 3- (special case) there is no shortest path from any vertex to any other that is ≥ 3									
	3.00	3.00	3.00	3.00	3.00	3.00	18.00	12.00	15.00

- a lot of components (e.g. 200)
sometimes there are bridges
average degree - 5-8

there is one really big component, so it can be treated as 3
- connected graph
no bridges
probably dense

average degree - less than 10% of N
higher than 11 (can possibly be treated as 7)
- connected graph
no bridges
normal average degree 5-6 - not really dense
- connected graph
no bridges
degree 2 - completely not dense
it is a structure similar to ring

around 0.6% (600) vertices have degree > 2
- connected graph
over 50% of edges are bridges
degree 2 - completely not dense
it is a structure similar to tree

half of test cases are below 75%
- connected graph
no bridges
there are no any shortest paths ≥ 3
it is a structure similar to complete graph

test33 has 222 avg degree and 1000 nodes
- connected graph
no bridges
average degree about 15

Solution for each graph type

Each type was compressed in the same way because we found it easier to manage later, even if it does not do much

1. Type 1:

- a. Graph sorting based on weights
- b. Heuristic-based search to 3.5 second of program run
- c. Local search to 19.3 second of program run
 - i. Alternative tail not random and not using backtrack
 - ii. Alternative tail not random and using backtrack
 - iii. Detour random, reversed and with max depth equal to 25000
 - iv. Extension

2. Type 2:

- a. Graph sorting based on weights
- b. Heuristic-based search to 3.5 second of program run
- c. Local search to 19.0 second of program run
 - i. Alternative tail not random and not using backtrack
 - ii. Alternative tail not random and using backtrack
 - iii. Detour random, reversed and with max depth equal to 10000
 - iv. Extension

3. Type 3:

- a. Default graph sorting
- b. Backtrack to 6.5 second of program run
- c. Local search to 19.3 second of program run
 - i. Alternative tail not random and not using backtrack
 - ii. Alternative tail random and not using backtrack
 - iii. Detour random, reversed and with max depth equal to 9000
 - iv. Extension

4. Type 4:

- a. Graph sorting based on weights
- b. Backtrack to 15.5 second of program run
- c. Local search to 19.4 second of program run
 - i. Alternative tail not random and not using backtrack
 - ii. Alternative tail random and not using backtrack
 - iii. Detour not random, reversed and with max depth equal to 10000
 - iv. Extension

5. Type 5:

- a. Graph sorting based on weights
- b. Backtrack to 16.5 second of program run
- c. Local search to 19.4 second of program run
 - i. Alternative tail not random and using backtrack
 - ii. Alternative tail random and not using backtrack
 - iii. Detour random, reversed and with max depth equal to 1000
 - iv. Extension

6. Type 6:

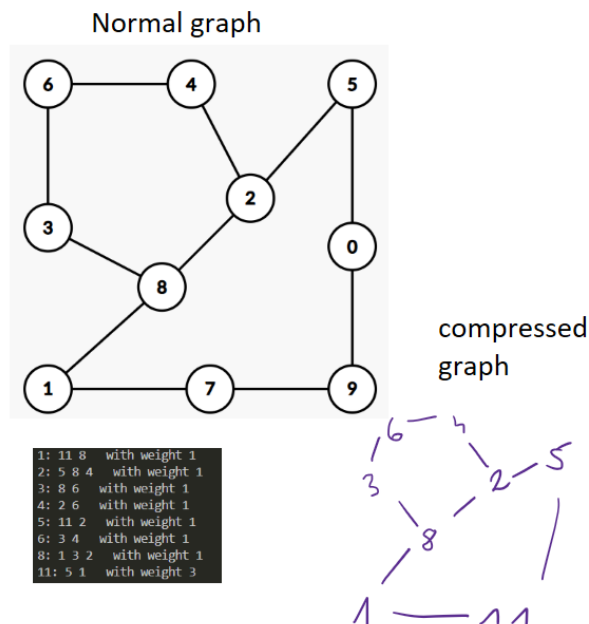
- a. Heuristic-based search to 3.5 second of program run
- b. Local search to 19.1 second of program run
 - i. Alternative tail not random and not using backtrack
 - ii. Alternative tail not random and using backtrack
 - iii. Detour random, reversed and with max depth equal to 100
 - iv. Extension

7. Type 7:

- a. Heuristic-based search to 3.5 second of program run
- b. Local search to 19.1 second of program run
 - i. Alternative tail not random and not using backtrack
 - ii. Alternative tail random and not using backtrack
 - iii. Detour not random, reversed and with max depth equal to 20000
 - iv. Extension

Graph compression

We compress graph based on „branches” which are nodes that have 2 at most 2 neighbors. To maintain properties of the original graph we compress only branches which all neighbors are also branches. Based on them we create one new node (called compressed node or comp node) that has a weight – number of nodes that it absorbed and took place of those compressed branches. Other remain nodes get weight 1.



Backtrack

We start it at a random node and run it for some particular time. Its purpose is to find a first best path that will be then changed by local search.

Heuristic-based search (start search)

Heuristic approach assumes, that the best extension is the one which sees as little unseen nodes (nodes that haven't been seen by nodes from path) as possible (at least one).

Local search (main search) – the idea

In local search we run different operations for as long as we can based on graph types. We check for alternative tail – a new part of the path that goes from our best path to some unseen node, detour – a new part of the path that goes from some node on our best path to the another and extension – extending this path as much as possible.

Local search (main search) – alternative tail

For each node from the path we start search for a new path (tail), it can't cross any previous part of the path. When extension of tail isn't further possible we check if it's beneficial to exchange it with left or right tail (nothing fancy, just by lengths of these tails).

Local search (main search) – detour

For each node from the path we look for a detour. We extend such detour and after each extension we check if it's possible to connect it back to the main path, then we check if such exchange is beneficial by looking up prefix values.

Local search (main search) – extension

Extends ends of current path as far as possible using heuristic and random approaches.

Decompression

Our last important part of the code is decompression. It is not always necessary as some compressed graphs (comp graphs) have no added nodes. We use data structures created in the compression step and recreate the final path

Important data structures

vector<vector<int>> graph – original graph

vector<vector<int>> comp_graph – compressed graph with new nodes (comp nodes)

vector<int> which_component – which node has which graph component

vector<int> which_comp – which node belongs to which compressed node

vector<int> weights – weight of each node (number of nodes that it absorbed during compression)

list<int> final_path – we used list for final path, because it was the fastest data structure for this kind of problem (we could add new nodes wherever we wanted in $O(1)$)

vector<CompNode> comp_nodes – vector that keeps newly added nodes (comp nodes) and information about them

vector<int> seen – a lookup table that stores for each node number of neighbours that belong to the path

vector<bool> vis – a lookup table that's true if given node is in current path and false otherwise

Good paths long

What was changed

We added new time limits for each type that were about 9 times larger than those before in short version.

Responsibilities

Most of ideas we created in cooperation, so these responsibilities are not that straightforward.

Joanna Zubik

I did data analysis, graph compression, decompression and backtracking (it sums up to about half of the code) and most of this documentation except of heuristic-based search and local search explanation.

Jędrzej Kolski

He did heuristic-based search, local search, set time limits and helped with backtracking and documentation.